ВВЕДЕНИЕ

В современном обществе благотворительная деятельность играет важную роль в оказании помощи социально незащищённым категориям населения, в частности детям, нуждающимся в медицинской, социальной или материальной поддержке. Одной из основных проблем благотворительных организаций является обеспечение прозрачности финансовых операций и поддержание доверия со стороны жертвователей. Потенциальные доноры не всегда имеют возможность отслеживать распределение пожертвованных средств, что может снижать уровень доверия к благотворительным фондам.

Развитие цифровых технологий открывает новые возможности для повышения прозрачности финансовых операций. Одним из перспективных решений является использование технологии блокчейн. Блокчейн представляет собой распределённый реестр данных, в котором информация о транзакциях хранится в виде взаимосвязанных блоков и защищается криптографическими методами. После подтверждения записи она становится неизменяемой, что исключает возможность её последующего изменения.

Использование блокчейн-технологии в системах учёта пожертвований позволяет создать надёжную инфраструктуру для регистрации финансовых операций. Каждая транзакция фиксируется в распределённой сети, что обеспечивает прозрачность и возможность отслеживания движения средств.

Целью данной дипломной работы является разработка *Web*-приложения для учёта пожертвований в детский благотворительный фонд с использованием технологии блокчейн, которое обеспечит прозрачность финансовых операций и безопасность хранения данных.

Для достижения поставленной цели необходимо решить следующие задачи:

– изучить особенности разработки современных *Web*-приложений и клиент-серверных архитектур;

– провести анализ технологии блокчейн и возможностей её применения;

– разработать архитектуру *Web*-приложения для учёта пожертвований;

– спроектировать структуру базы данных и механизм взаимодействия с блокчейн-сетью;

– реализовать основные программные модули системы;

– провести тестирование разработанного программного обеспечения.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Анализ предметной области

Анализ предметной области «*Web*-приложение для учета пожертвований в детский фонд с обеспечением прозрачности операций» позволяет определить основные процессы и требования, которые должны быть реализованы в разрабатываемом *Web*-приложении. Благотворительные организации осуществляют сбор финансовых средств для поддержки различных социальных проектов, однако одной из ключевых проблем в данной сфере является обеспечение прозрачности операций и повышение доверия со стороны доноров.

Одним из важнейших аспектов является процесс сбора пожертвований. Пользователи должны иметь возможность быстро и удобно осуществлять перевод средств на поддержку благотворительных проектов. В системе могут поддерживаться различные виды пожертвований, включая разовые переводы, регулярные платежи по подписке, целевые пожертвования на конкретные проекты, а также переводы в общий фонд организации. Возможность выбора различных способов оплаты (банковские карты, электронные кошельки, банковские переводы и криптовалюты) позволяет сделать процесс пожертвования максимально доступным для различных категорий пользователей.

Другим важным аспектом является обеспечение прозрачности финансовых операций. Потенциальные доноры должны иметь возможность отслеживать, каким образом используются пожертвованные средства. Для этого система должна предоставлять открытую статистику поступлений и расходов, отображаемую в виде графиков, диаграмм и аналитических отчётов. Также пользователи должны иметь доступ к информации о каждом благотворительном проекте, включая его описание, цели, текущий прогресс сбора средств, а также отчёты о расходовании средств.

Значительную роль играет управление благотворительными проектами. Администраторы системы должны иметь возможность создавать и редактировать проекты, указывать цели сбора средств, сроки реализации, описание проекта и ответственных лиц. Кроме того, система должна обеспечивать возможность ведения учёта расходов, включая добавление финансовых операций и загрузку подтверждающих документов, таких как чеки, счета и отчётные материалы.

Важным элементом системы является личный кабинет пользователя, который предоставляет донорам возможность отслеживать историю своих пожертвований, получать уведомления о поступлении средств и ходе реализации проектов, а также настраивать регулярные платежи. Наличие такой функциональности повышает удобство использования системы и способствует формированию долгосрочного взаимодействия между пользователями и благотворительным фондом.

Отдельного внимания требует система уведомлений и обратной связи с пользователями. После совершения пожертвования пользователь должен получать подтверждение операции, благодарственное сообщение, а также информа-

цию о том, каким образом были использованы переданные средства. Это позволяет повысить уровень доверия к благотворительной организации и стимулирует пользователей к повторному участию в благотворительных инициативах.

Для повышения активности пользователей может быть реализована система мотивации доноров, включающая рейтинги, достижения и уровни участия. Например, пользователи могут получать виртуальные награды или статусы в зависимости от суммы и регулярности пожертвований. Подобный механизм способствует формированию сообщества активных доноров и повышает вовлечённость пользователей.

Одной из ключевых задач разрабатываемой системы является обеспечение прозрачности и достоверности данных о финансовых операциях. Для решения данной задачи может быть использована технология блокчейн, которая позволяет хранить информацию о транзакциях в распределённом реестре. Каждая операция записывается в блокчейн и защищается криптографическими методами, что исключает возможность изменения или удаления данных задним числом.

Использование блокчейн-технологии позволяет создать надёжную систему учёта пожертвований, в которой каждый пользователь сможет проверить факт поступления средств и проследить их дальнейшее использование. Такой подход значительно повышает уровень доверия к благотворительным организациям и минимизирует риски неправомерного использования средств.

Разработка *Web*-приложения для сбора и учёта пожертвований позволит автоматизировать основные процессы благотворительной деятельности, обеспечить прозрачность финансовых операций и создать удобный инструмент взаимодействия между благотворительным фондом и донорами. Реализация подобной системы может значительно повысить эффективность управления благотворительными проектами и способствовать развитию культуры благотворительности.

1.2 Аналитический обзор существующих аналогов

В настоящее время существует множество информационных систем и онлайн-платформ, предназначенных для организации благотворительных сборов и управления пожертвованиями. Такие решения позволяют благотворительным организациям привлекать финансовые средства, информировать пользователей о реализуемых проектах и обеспечивать взаимодействие с донорами. Использование современных *Web*-технологий делает процесс пожертвования более удобным и доступным для широкого круга пользователей.

Одним из наиболее простых способов организации благотворительных сборов является использование традиционных методов, таких как банковские переводы, наличные пожертвования или сбор средств через отдельные платежные формы на сайтах организаций. Однако подобные методы часто имеют ограниченные возможности для анализа и отслеживания движения средств. Пользователи не всегда могут получить подробную информацию о том, каким

образом были использованы пожертвованные средства, что может снижать уровень доверия к благотворительным организациям.

В последние годы активно развиваются специализированные онлайн-платформы для благотворительности. Такие системы предоставляют удобные инструменты для размещения проектов, сбора пожертвований и публикации отчетов о расходовании средств. Как правило, подобные платформы включают каталог благотворительных проектов, систему онлайн-платежей, статистику пожертвований и инструменты взаимодействия с пользователями.

Одним из известных сервисов является платформа *GoFundMe*, которая позволяет пользователям создавать страницы для сбора средств на различные цели, включая медицинскую помощь, социальные инициативы и благотворительные проекты. Пользователи могут публиковать описание проекта, устанавливать цель сбора средств и отслеживать текущий прогресс кампании. Платформа предоставляет возможность принимать пожертвования с использованием банковских карт и электронных платежных систем. Кроме того, система позволяет делиться страницами проектов через социальные сети, что способствует более широкому распространению информации о благотворительных инициативах. Однако одним из недостатков подобных сервисов является ограниченная прозрачность финансовых операций, так как пользователи не всегда имеют возможность проследить полный путь движения средств после их перечисления.

Другим примером является благотворительная платформа *JustGiving*, которая также предоставляет инструменты для создания кампаний по сбору средств и взаимодействия с донорами. Пользователи могут просматривать проекты, совершать пожертвования и отслеживать статистику собранных средств. Платформа поддерживает различные способы оплаты и предоставляет организаторам инструменты аналитики и отчетности. Несмотря на развитый функционал, такие системы остаются централизованными, что означает, что контроль за финансовыми операциями осуществляется администрацией сервиса.

В последние годы появляются решения, использующие технологию блокчейн для повышения прозрачности благотворительных операций. Блокчейн позволяет фиксировать информацию о транзакциях в распределённом реестре, что делает данные неизменяемыми и доступными для проверки. Использование данной технологии позволяет повысить уровень доверия пользователей, поскольку каждая финансовая операция может быть подтверждена и проверена в сети.

Несмотря на наличие различных платформ для организации благотворительных сборов, многие из них имеют ряд недостатков. К основным проблемам можно отнести недостаточную прозрачность использования средств, ограниченные возможности отслеживания транзакций, а также зависимость от централизованных систем управления. В связи с этим актуальной задачей является разработка *Web*-приложения для сбора и учёта пожертвований, которое будет обеспечивать высокий уровень прозрачности финансовых операций за счёт использования технологии блокчейн.

1.3 Обзор технологий для реализации программного обеспечения

Современная разработка *Web*-приложений основывается на использовании различных программных технологий и инструментов, позволяющих создавать масштабируемые, надёжные и безопасные информационные системы. Для реализации системы сбора и учёта пожертвований с обеспечением прозрачности финансовых операций был выбран комплекс технологий, включающий клиентскую часть приложения, серверную часть, систему управления базами данных, а также технологии блокчейн.

Архитектура разрабатываемой системы основана на клиент-серверной модели, при которой пользователь взаимодействует с интерфейсом *Web*-приложения через браузер. Клиентская часть приложения отвечает за отображение интерфейса и обработку пользовательских действий, тогда как серверная часть обеспечивает обработку запросов, хранение данных и реализацию бизнес-логики. Взаимодействие между компонентами системы осуществляется через сетевые протоколы передачи данных и специализированные интерфейсы программного взаимодействия.

JavaScript является одним из наиболее распространённых языков программирования для разработки *Web*-приложений. Он представляет собой мультипарадигменный язык программирования, поддерживающий объектно-ориентированный, функциональный и императивный стили разработки.

JavaScript используется для создания интерактивных пользовательских интерфейсов, обработки событий и динамического изменения содержимого веб-страниц. Основные особенности языка включают динамическую типизацию, автоматическое управление памятью и поддержку асинхронного программирования.

В современных Web-приложениях JavaScript активно используется для обмена данными между клиентом и сервером с применением технологии *AJAX*, которая позволяет выполнять асинхронные запросы без полной перезагрузки страницы. Благодаря этому пользовательский интерфейс становится более быстрым и удобным.

React является популярной библиотекой *JavaScript* для разработки пользовательских интерфейсов. Она была разработана компанией *Meta* (*Facebook*) и широко применяется при создании современных Web-приложений.

На рисунке 1.1 представлена схема взаимодействия *React* с *Virtual DOM.*

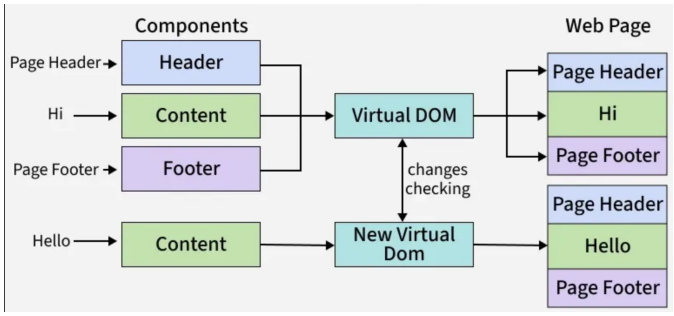

Рисунок 1.1 — Схема взаимодействия *React* с *Virtual DOM*

Основной особенностью *React* является компонентный подход к разработке интерфейсов. Пользовательский интерфейс разбивается на отдельные компоненты, каждый из которых отвечает за определённую часть функциональности приложения. Такой подход упрощает разработку, повышает повторное использование кода и облегчает поддержку программного обеспечения.

React использует механизм *Virtual DOM*, который позволяет оптимизировать обновление интерфейса. Вместо полного обновления веб-страницы система обновляет только те элементы, которые действительно изменились. Это значительно повышает производительность приложения.

Использование *React* позволяет создавать динамические и удобные интерфейсы, что особенно важно для систем, работающих с большим количеством пользовательских данных и интерактивных элементов.

Для разработки серверной части приложения используется платформа *ASP.NET*, которая является частью экосистемы *Microsoft .NET*. Данная платформа предназначена для создания *Web*-приложений, *API*-сервисов и распределённых информационных систем.

Основным языком разработки на платформе *ASP.NET* является *C#* — современный объектно-ориентированный язык программирования со строгой типизацией. Он обладает высокой производительностью, развитой системой управления памятью и поддерживает современные парадигмы программирования.

ASP.NET предоставляет разработчикам широкий набор инструментов для реализации серверной логики, включая обработку *HTTP*-запросов, управление пользовательскими данными, реализацию механизмов безопасности и взаимодействие с базами данных.

Одним из важных преимуществ *ASP.NET* является возможность создания *REST API*, которое обеспечивает взаимодействие между клиентской и серверной частями приложения.

REST API (*Representational State Transfer Application Programming Interface*) представляет собой архитектурный стиль взаимодействия программных систем через сеть. Он широко используется в *Web*-разработке для организации обмена данными между клиентскими и серверными приложениями.

В рамках *REST API* взаимодействие между компонентами системы осуществляется с использованием стандартных *HTTP*-методов, таких как:

– *GET* — получение данных;

– *POST* — создание новых записей;

– *PUT* — обновление существующих данных;

– *DELETE* — удаление данных.

Передача данных между клиентом и сервером осуществляется в формате *JSON*, который является удобным и широко распространённым форматом обмена информацией.

Использование *REST API* позволяет разделить клиентскую и серверную части приложения, обеспечивая гибкость архитектуры и возможность масштабирования системы.

Для хранения данных используется система управления базами данных *PostgreSQL*, которая является одной из наиболее популярных реляционных СУБД с открытым исходным кодом.

PostgreSQL обеспечивает высокую надёжность хранения данных, поддержку транзакций и возможность обработки сложных запросов. Система поддерживает язык структурированных запросов *SQL*, который используется для создания, изменения и извлечения данных из базы.

В рамках разрабатываемой системы база данных используется для хранения информации о пользователях, благотворительных проектах, транзакциях пожертвований, а также финансовых отчётах.

Для обеспечения безопасности доступа к системе используется механизм аутентификации *JWT* (*JSON Web Token*).

JWT представляет собой компактный токен, который используется для безопасной передачи информации между клиентом и сервером. После успешной авторизации пользователя сервер формирует токен, содержащий зашифрованную информацию о пользователе и его правах доступа.

Этот токен сохраняется на стороне клиента и передаётся серверу при выполнении последующих запросов. Сервер проверяет подпись токена и предоставляет доступ к ресурсам системы только авторизованным пользователям.

Использование *JWT* позволяет реализовать эффективную и безопасную систему авторизации без необходимости хранения пользовательских сессий на сервере.

Для обеспечения прозрачности финансовых операций в системе используется технология блокчейн на базе сети *Ethereum*. *Ethereum* является децентрализованной платформой, предназначенной для создания и выполнения смарт-контрактов.

В сети *Ethereum* информация о транзакциях хранится в распределённом реестре, который состоит из последовательности блоков. Каждый блок содержит набор транзакций и связан с предыдущим блоком с использованием криптографических методов.

После добавления данных в блокчейн изменить или удалить их невозможно, что обеспечивает высокий уровень безопасности и достоверности информации.

Использование сети *Ethereum* позволяет фиксировать операции пожертвований в распределённом реестре, что обеспечивает прозрачность финансовых операций и повышает доверие пользователей к системе.

Solidity является языком программирования, предназначенным для разработки смарт-контрактов в сети *Ethereum*.

Смарт-контракты представляют собой программы, которые автоматически выполняются в блокчейн-сети при наступлении определённых условий. Они позволяют реализовать различные сценарии обработки транзакций без участия третьих сторон.

В разрабатываемой системе смарт-контракты используются для регистрации пожертвований, хранения информации о транзакциях и обеспечения прозрачности финансовых операций. После публикации смарт-контракта в блок-

чейне его код становится неизменяемым, что гарантирует соблюдение установленных правил работы системы.

Для взаимодействия пользователей с блокчейн-сетью используется криптовалютный кошелёк *MetaMask*. Он представляет собой браузерное расширение, которое позволяет пользователям хранить криптовалюту, управлять цифровыми активами и взаимодействовать с децентрализованными приложениями.

MetaMask обеспечивает подключение пользовательского кошелька к *Web*-приложению и позволяет подтверждать транзакции в сети *Ethereum*. При совершении пожертвования пользователь подписывает транзакцию с помощью своего приватного ключа, после чего операция отправляется в блокчейн-сеть.

Использование *MetaMask* позволяет обеспечить безопасное взаимодействие пользователей с системой и исключить необходимость хранения приватных ключей на сервере приложения.

Для взаимодействия *Web*-приложения с блокчейн-сетью используются специализированные библиотеки *Web3.js* или *Ethers.js*.

Эти библиотеки позволяют приложениям отправлять транзакции в сеть *Ethereum*, получать информацию о состоянии блокчейна и взаимодействовать со смарт-контрактами.

С их помощью клиентская часть приложения может выполнять такие операции, как:

– подключение к криптовалютному кошельку пользователя;

– отправка транзакций пожертвований;

– получение информации о выполненных операциях;

– чтение данных из смарт-контрактов.

Использование данных библиотек обеспечивает удобную интеграцию *Web*-приложения с блокчейн-инфраструктурой.

MinIO — это высокопроизводительное объектное хранилище с открытым исходным кодом, разработанное специально для облачной инфраструктуры и работы с большими данными . Оно распространяется по лицензии *GNU AGPL v3*, что позволяет использовать его в образовательных и коммерческих проектах с определенными условиями . Ключевым преимуществом *MinIO* является его полная совместимость с *Amazon S3 API*, что делает его стандартом де-факто для *on-premise* решений, позволяя использовать существующие инструменты и библиотеки экосистемы *S3* без изменений .

Структура хранения данных строится следующим образом:

– *Buckets* (Корзины): в *MinIO* создаются логические разделы — корзины, например, корзина *user-avatars* для хранения аватарок пользователей и *product-images* для изображений товаров.

– *Objects* (Объекты): сами файлы сохраняются как объекты, для обеспечения уникальности и кеширования, имя файла на сервере генерируется как хеш-сумма от оригинального имени и временной метки, это позволяет избежать коллизий и гарантирует, что при обновлении файла будет создана новая версия

1.4 Анализ инструментальных средств автоматизации разработки и тестирования

Для разработки *Web*-приложения сбора и учёта пожертвований используется ряд современных программных инструментов, которые позволяют автоматизировать процесс разработки, тестирования и развертывания программного обеспечения. Использование специализированных инструментов повышает эффективность разработки, упрощает управление проектом и обеспечивает более высокое качество создаваемого программного продукта.

В рамках реализации дипломного проекта используются следующие инструментальные средства: интегрированная среда разработки *JetBrains Rider*, система контейнеризации *Docker*, система контроля версий *Git* и платформа *GitHub*, инструмент работы с базами данных *DataGrip*, средство тестирования *API Postman*, а также инструменты разработки смарт-контрактов и взаимодействия с блокчейн-сетью.

JetBrains Rider является современной интегрированной средой разработки (*IDE*), предназначенной для создания приложений на платформе *.NET*. Она разработана компанией *JetBrains* и предоставляет широкий набор инструментов для разработки программного обеспечения на языке программирования *C#*.

Среда разработки *Rider* обеспечивает поддержку разработки серверной части *Web*-приложений на платформе *ASP.NET*, включая создание *REST API*, работу с базами данных и реализацию бизнес-логики приложения.

Основные возможности *JetBrains Rider* включают:

– интеллектуальное автодополнение кода;
– встроенные инструменты отладки;
– анализ кода и поиск ошибок;
– интеграцию с системой контроля версий *Git*;
– поддержку работы с контейнерами *Docker*;
– инструменты рефакторинга кода.

Одним из преимуществ *Rider* является высокая производительность и удобный интерфейс, позволяющий разработчику эффективно работать с большими программными проектами. Кроме того, среда разработки поддерживает большое количество плагинов, расширяющих её функциональность.

Docker является популярной платформой для контейнеризации приложений. Она позволяет запускать программное обеспечение в изолированных контейнерах, которые содержат все необходимые зависимости и настройки среды выполнения.

Использование *Docker* позволяет значительно упростить процесс развертывания и тестирования приложения, поскольку контейнеры могут запускаться одинаково на различных операционных системах и серверах.

Основные преимущества *Docker* включают:

– изоляцию приложений и их зависимостей;
– упрощение процесса развертывания программного обеспечения;
– возможность масштабирования системы;
– повышение стабильности среды выполнения.

В рамках разрабатываемого проекта *Docker* используется для контейнеризации серверной части приложения и базы данных, что позволяет обеспечить стабильность работы системы и упростить процесс её развертывания.

Для управления исходным кодом используется распределённая система контроля версий *Git*. Она позволяет отслеживать изменения в коде, сохранять историю разработки и возвращаться к предыдущим версиям программного обеспечения при необходимости.

Для удалённого хранения репозитория используется платформа *GitHub*, которая предоставляет разработчикам возможность хранить код проекта в облаке и работать над ним совместно.

Основные возможности *Git* и *GitHub* включают:

– хранение и управление версиями программного кода;

– создание веток разработки;

– объединение изменений (*merge*);

– отслеживание истории изменений;

– совместную работу нескольких разработчиков.

Использование *GitHub* позволяет избежать потери данных и обеспечивает безопасное хранение исходного кода проекта.

DataGrip является инструментом для работы с базами данных, разработанным компанией *JetBrains*. Он предоставляет удобный графический интерфейс для управления базами данных и выполнения *SQL*-запросов.

DataGrip поддерживает большое количество систем управления базами данных, включая *PostgreSQL*, которая используется в рамках данного проекта.

Основные возможности *DataGrip* включают:

– выполнение *SQL*-запросов;

– визуальное управление таблицами базы данных;

– анализ структуры базы данных;

– автодополнение *SQL*-кода;

– просмотр и редактирование данных.

Использование *DataGrip* позволяет значительно упростить процесс разработки и администрирования базы данных приложения.

Postman является популярным инструментом для тестирования *API*-интерфейсов. Он позволяет отправлять *HTTP*-запросы к серверу и анализировать полученные ответы.

С помощью *Postman* разработчики могут тестировать работу *REST API*, проверять корректность обработки запросов и анализировать данные, возвращаемые сервером.

Основные возможности *Postman* включают:

– отправку *HTTP*-запросов различных типов (*GET*, *POST*, *PUT*, *DELETE*);

– передачу параметров и данных в формате *JSON*;

– тестирование авторизации и аутентификации;

– создание коллекций запросов;

– автоматизацию тестирования *API*.

На рисунке 1.2 представлен пользовательский интерфейс *Postman*.

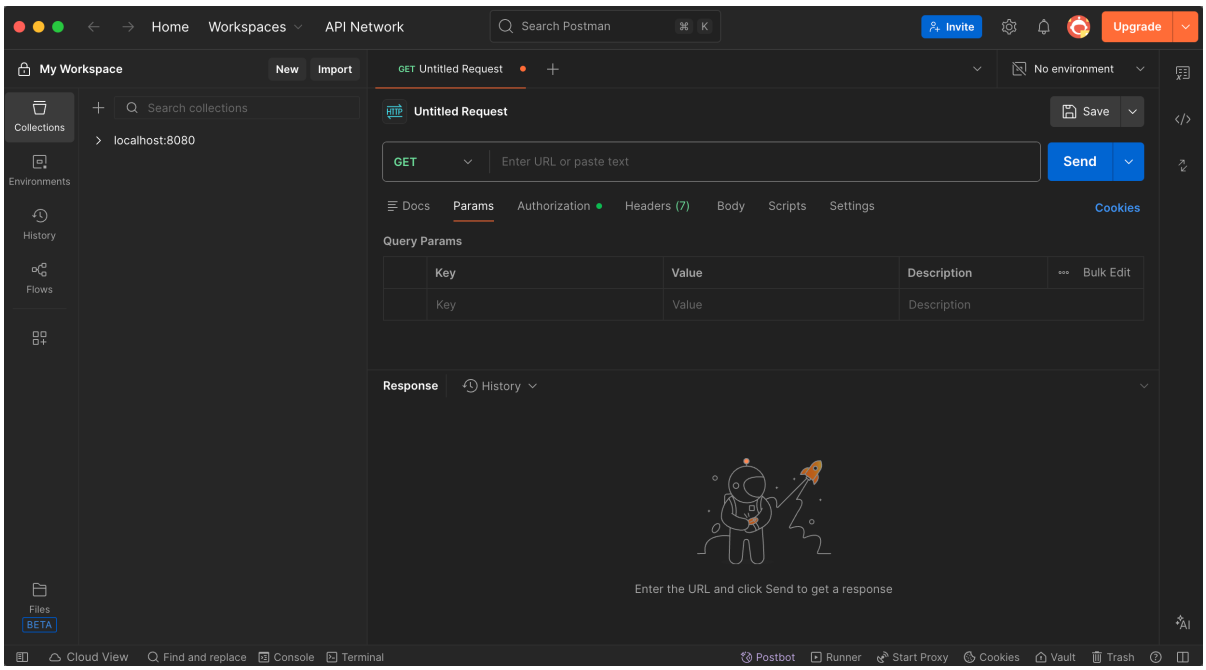

Рисунок 1.2 — Интерфейс главного окна *Postman*

Использование *Postman* позволяет значительно ускорить процесс разработки и тестирования серверной части *Web*-приложения.

MetaMask является криптовалютным кошельком, который используется для взаимодействия пользователей с блокчейн-сетью *Ethereum*. Он представляет собой браузерное расширение, позволяющее хранить криптовалюту и взаимодействовать с децентрализованными приложениями.

В рамках разрабатываемого проекта *MetaMask* используется для тестирования и подтверждения транзакций пожертвований в сети *Ethereum*. Пользователь подписывает транзакцию с помощью своего приватного ключа, после чего она отправляется в блокчейн-сеть.

Использование *MetaMask* позволяет безопасно взаимодействовать с блокчейн-инфраструктурой без необходимости хранения приватных ключей на сервере.

Hardhat является современным инструментом для разработки, тестирования и развертывания смарт-контрактов в сети *Ethereum*. Данный инструмент представляет собой среду разработки, предназначенную для создания децентрализованных приложений (*DApp*).

Hardhat предоставляет разработчикам набор инструментов, позволяющих автоматизировать процесс разработки смарт-контрактов, включая компиляцию контрактов, запуск локальной блокчейн-сети, тестирование и развертывание контрактов в различных сетях *Ethereum*.

Основные возможности *Hardhat* включают:

– компиляцию смарт-контрактов на языке *Solidity*;
– запуск локальной сети *Ethereum* для тестирования;
– автоматическое тестирование контрактов;
– развертывание смарт-контрактов в тестовых и публичных сетях;
– отладку транзакций и контрактов.

Одним из ключевых преимуществ *Hardhat* является возможность использования локальной тестовой сети, что позволяет разработчикам тестировать работу смарт-контрактов без необходимости выполнения реальных транзакций в публичной сети *Ethereum*.

Использование *Hardhat* значительно упрощает процесс разработки и тестирования блокчейн-компонентов системы.

draw.io является инструментом для создания различных типов диаграмм, используемых при проектировании программных систем. Данный инструмент позволяет визуализировать архитектуру приложения, структуру базы данных, а также взаимодействие компонентов системы.

С помощью *draw.io* можно создавать следующие типы диаграмм:

– *UML*-диаграммы;

– диаграммы компонентов;

– диаграммы последовательностей;

– диаграммы бизнес-процессов;

– архитектурные схемы системы.

Одним из преимуществ draw.io является простой и интуитивно понятный интерфейс, который позволяет быстро создавать графические схемы и экспортировать их в различные форматы, такие как *PNG*, *PDF* или *SVG*.

В рамках дипломного проекта данный инструмент используется для построения архитектурных схем системы и *UML*-диаграмм, которые позволяют наглядно представить структуру разрабатываемого *Web*-приложения.

Для тестирования работы смарт-контрактов и выполнения транзакций используется тестовая сеть *Ethereum* (*Testnet*). Тестовые сети предназначены для разработки и проверки работы децентрализованных приложений без использования реальных криптовалютных средств.

В отличие от основной сети *Ethereum*, тестовые сети позволяют разработчикам выполнять транзакции бесплатно, используя специальные тестовые токены.

В рамках разработки *Web*-приложения для учета пожертвований используется тестовая сеть *Sepolia*, которая является одной из наиболее распространенных тестовых сетей *Ethereum*.

Использование тестовой сети позволяет:

– тестировать выполнение смарт-контрактов;

– проверять корректность обработки транзакций;

– анализировать взаимодействие приложения с блокчейном;

– выявлять ошибки в работе системы до её развертывания в основной сети.

Применение тестовой сети *Ethereum* является важным этапом разработки блокчейн-приложений, поскольку позволяет обеспечить стабильность и безопасность работы системы.

1.5 Постановка задачи на дипломное проектирование

Цель разработки — создание *Web*-приложения для сбора и учёта пожертвований в детский благотворительный фонд с обеспечением прозрачности финансовых операций с использованием технологии блокчейн.

Разрабатываемый программный комплекс предназначен для организации процесса пожертвований, повышения прозрачности финансовых потоков и повышения доверия со стороны пользователей и благотворителей. Использование технологии блокчейн позволяет фиксировать информацию о транзакциях в распределённой сети, что обеспечивает неизменяемость и проверяемость данных о совершённых пожертвованиях.

Основными пользователями системы являются:

– гости системы;

– зарегистрированные пользователи (доноры);

– администраторы фонда.

Для гостей системы приложение предусматривает следующие функции:

– регистрация и авторизация в системе;

– просмотр информации о благотворительном фонде и его деятельности;

– просмотр открытой финансовой статистики;

– просмотр списка благотворительных проектов;

– просмотр частичной информации о финансовых отчётах.

Для зарегистрированных пользователей приложение предусматривает следующие функции:

– создание и редактирование пользовательского профиля;

– просмотр личной истории пожертвований;

– настройка регулярных пожертвований;

– получение уведомлений о проведённых операциях и обновлениях проектов;

– просмотр отчётов о расходовании пожертвованных средств.

Для администратора системы приложение предусматривает следующие функции:

– управление пользователями системы;

– создание, редактирование и удаление благотворительных проектов;

– управление информацией о проектах;

– публикация финансовых отчётов;

– контроль поступлений и распределения средств.

Основной функционал системы включает реализацию механизма пожертвований, который поддерживает несколько типов финансовых операций:

– разовые пожертвования;

– регулярные пожертвования (подписка);

– целевые пожертвования на конкретный проект;

– свободные пожертвования в общий фонд.

Пользователь должен иметь возможность выбирать сумму пожертвования, а также способ оплаты. В качестве способов оплаты могут использоваться банковские карты, а также криптовалютные транзакции.

Система должна обеспечивать высокий уровень прозрачности и отчётности. Для этого реализуется публичный интерфейс со статистикой поступлений средств, распределением средств по проектам и динамикой пожертвований. Для каждого проекта должна отображаться информация о цели сбора средств, текущей сумме пожертвований, а также отчёты о расходовании средств.

Кроме того, приложение должно предоставлять личный кабинет пользователя, в котором отображается история всех пожертвований, статус финансовых операций, настройки уведомлений и возможность управления пользовательским профилем.

Административная часть системы должна обеспечивать возможность управления благотворительными проектами, ведения учёта расходов, добавления подтверждающих документов и публикации результатов реализации проектов.

Разрабатываемое приложение должно обладать следующими характеристиками:

– функционировать как *Web*-приложение;

– использовать клиент-серверную архитектуру;

– обеспечивать взаимодействие клиентской и серверной частей посредством *REST API*;

– обеспечивать интеграцию с блокчейн-сетью *Ethereum* для регистрации транзакций пожертвований;

– быть кроссплатформенным;

– быть кроссбраузерным;

– обеспечивать безопасность хранения и передачи данных пользователей;

– предоставлять удобный пользовательский интерфейс.

2 АРХИТЕКТУРА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

2.1 Общая структура приложения

Архитектура программного обеспечения разрабатываемого *Web*-приложения зависит от множества факторов, включая масштаб системы, требования к функциональности, безопасности, производительности и возможности дальнейшего масштабирования. В рамках данной работы разрабатывается *Web*-приложение благотворительного фонда, предназначенное для организации процесса сбора пожертвований, управления благотворительными проектами и обеспечения прозрачности финансовых операций.

На рисунке 2.1 представлена архитектура *Web*-приложения.

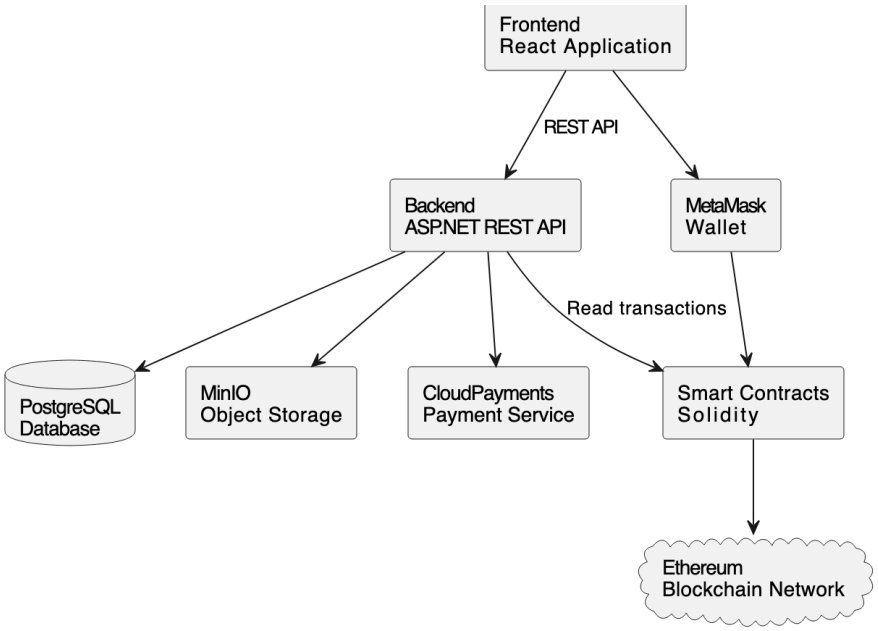

Рисунок 2.1 — Архитектура *Web*-приложения

Общая структура приложения построена на основе клиент-серверной архитектуры. В данной архитектуре система разделяется на несколько логических уровней, каждый из которых выполняет определённые функции. Пользователь взаимодействует с системой через веб-браузер, используя клиентскую часть приложения, которая отвечает за отображение пользовательского интерфейса и обработку действий пользователя. Серверная часть системы выполняет обработку запросов, реализацию бизнес-логики приложения, взаимодействие с базой данных и внешними сервисами.

Клиентская часть приложения реализована с использованием библиотеки *React*, которая является одной из наиболее популярных технологий для разработки пользовательских интерфейсов *Web*-приложений. *React* использует компонентный подход, при котором интерфейс приложения разбивается на отдельные независимые компоненты. Каждый компонент отвечает за определённую

часть пользовательского интерфейса и реализует отдельную функциональность системы.

Компоненты *React* обычно создаются в файлах с расширением *.jsx* и представляют собой расширение синтаксиса *JavaScript*, позволяющее описывать структуру интерфейса с использованием элементов, похожих на *HTML*. Такой подход позволяет значительно упростить разработку интерфейса и повысить повторное использование программного кода.

На этапе проектирования приложение разделяется на множество компонентов, таких как страницы просмотра благотворительных проектов, страницы финансовой отчётности, формы пожертвований, панели управления пользователями и другие элементы интерфейса. Для обеспечения взаимодействия между компонентами используются механизмы *props* и *state*, которые позволяют передавать данные между компонентами и управлять состоянием интерфейса.

JavaScript является основным языком программирования, используемым для разработки клиентской части приложения. Он обеспечивает обработку пользовательских событий, динамическое обновление содержимого веб-страниц и взаимодействие с серверной частью приложения.

Серверная часть приложения реализована на платформе *ASP.NET*, которая является частью экосистемы *Microsoft .NET* и предназначена для разработки *Web*-приложений и *API*-сервисов. В качестве основного языка программирования используется *C#*, обеспечивающий высокую производительность и поддержку современных методов разработки программного обеспечения.

Серверная часть системы реализует бизнес-логику приложения, выполняет обработку пользовательских запросов, управляет данными пользователей, благотворительных проектов и пожертвований, а также обеспечивает взаимодействие с внешними сервисами.

Для организации взаимодействия между клиентской и серверной частями используется архитектурный стиль *REST API*. *REST API* представляет собой интерфейс программного взаимодействия, основанный на использовании стандартных *HTTP*-методов.

Основными методами *REST*-взаимодействия являются:

– *GET* — получение данных;

– *POST* — создание новых записей;

– *PUT* — обновление существующих данных;

– *DELETE* — удаление данных.

Передача данных между клиентом и сервером осуществляется в формате *JSON*, который является удобным и широко используемым форматом обмена информацией в *Web*-приложениях.

Для хранения основной информации системы используется система управления базами данных *PostgreSQL*. Данная СУБД обеспечивает надёжное хранение данных, поддержку транзакций и возможность выполнения сложных запросов. В базе данных хранятся сведения о пользователях системы, благотворительных проектах, совершённых пожертвованиях и финансовых отчётах.

Хранение файловых данных, таких как изображения благотворительных проектов, аватары пользователей и документы финансовой отчётности, осу-

ществляется с использованием объектного хранилища *MinIO*. Данная система обеспечивает эффективное хранение файлов и совместима с *API Amazon S3*, что позволяет удобно управлять файлами и обеспечивать доступ к ним из приложения.

Структура хранения файлов в *MinIO* организуется с использованием логических контейнеров — *bucket*, внутри которых файлы хранятся в виде объектов. Такой подход обеспечивает удобную организацию файловых данных и возможность масштабирования системы хранения.

Для реализации операций пожертвований используется интеграция с платёжным сервисом *CloudPayments*, который предоставляет *API* для обработки онлайн-платежей. Через данный сервис пользователи могут совершать пожертвования с использованием банковских карт, а информация о платежах передаётся в серверную часть приложения для дальнейшей обработки и хранения.

Дополнительно для обеспечения прозрачности финансовых операций в системе используется технология блокчейн на базе сети *Ethereum*. В рамках системы используются смарт-контракты, реализованные на *языке* программирования *Solidity*, которые позволяют фиксировать информацию о транзакциях пожертвований в распределённом реестре.

Смарт-контракты представляют собой программный код, который выполняется непосредственно в блокчейн-сети и автоматически обрабатывает транзакции при наступлении определённых условий. После размещения смарт-контракта в блокчейне изменить его код невозможно, что обеспечивает неизменяемость правил обработки транзакций.

Для взаимодействия пользователя с блокчейн-сетью используется криптовалютный кошелёк *MetaMask*, который позволяет пользователям подключаться к *Web*-приложению, подтверждать транзакции и отправлять их в сеть *Ethereum*.

Клиентская часть приложения взаимодействует со смарт-контрактами через специализированные библиотеки *Web3.js* или *Ethers.js*, которые позволяют отправлять транзакции, получать данные из блокчейн-сети и отслеживать состояние выполненных операций.

2.2 Функциональная модель программного комплекса

Для разработки программного комплекса необходимо определить пользователей системы и функциональные возможности, которые будут им доступны. Для этого используется функциональная модель предметной области, позволяющая наглядно представить взаимодействие пользователей с системой.

Функциональная модель описывает основные сценарии использования системы и роли пользователей, взаимодействующих с программным комплексом. Для построения функциональной модели применяется язык моделирования *UML* (*Unified Modeling Language*), позволяющий графически представить структуру и поведение системы.

Актёр в *UML* представляет собой пользователя или внешнюю систему, взаимодействующую с программным комплексом. Актёром может быть как че-

ловек, так и программная система, находящаяся вне границ рассматриваемого приложения.

Прецедент (use case) описывает определённый сценарий взаимодействия пользователя с системой, в результате которого достигается конкретная цель пользователя. Каждый прецедент представляет последовательность действий, выполняемых системой для получения пользователем определённого результата.

Использование диаграммы прецедентов позволяет определить:

– роли пользователей системы;

– основные функции программного комплекса;

– взаимодействие пользователей с системой.

В рамках разрабатываемого программного комплекса можно выделить следующие роли пользователей:

– гость системы;

– донор;

– администратор фонда.

Роль донор является расширением роли гость, поскольку зарегистрированный пользователь обладает всеми возможностями гостя, а также дополнительным функционалом, связанным с осуществлением пожертвований и управлением личным профилем.

Функциональная модель программного комплекса представлена на рисунке 2.2.

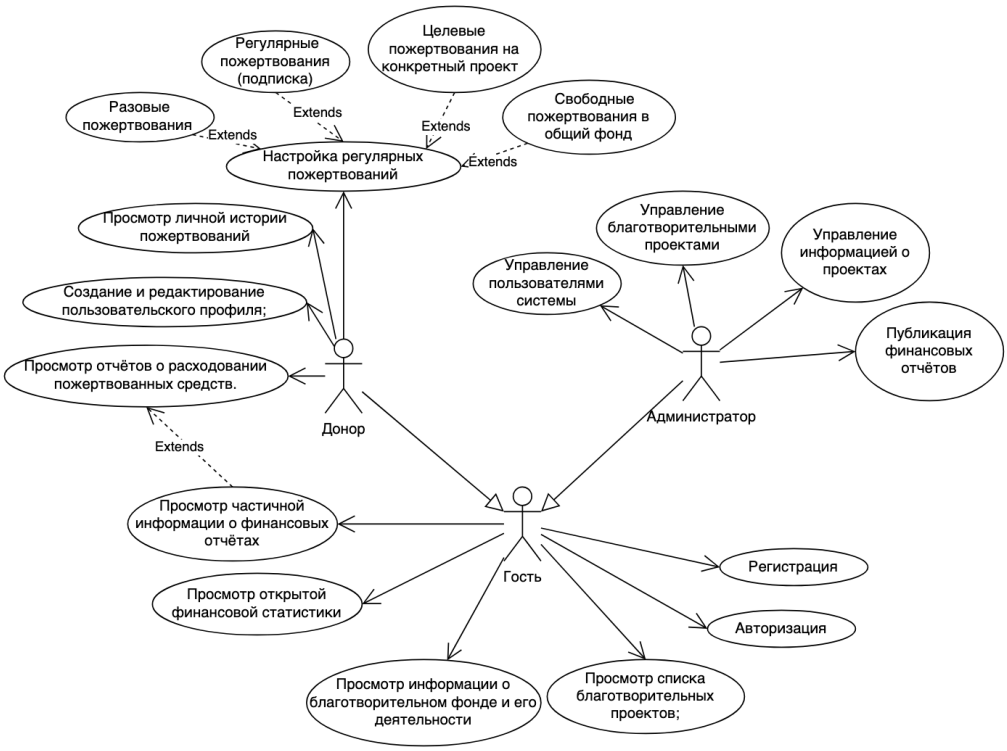

Рисунок 2.2 — Диаграмма прецедентов и актеров

Далее приведено описание прецедентов для роли «Гость».

Прецедент «Просмотр информации о благотворительном фонде и его деятельности» предназначен для ознакомления пользователя с общей информацией о фонде, его миссией, направлениями деятельности и текущими инициативами.

Прецедент «Просмотр списка благотворительных проектов» позволяет пользователю ознакомиться с перечнем проектов, реализуемых фондом. В рамках данного прецедента пользователь может получить краткую информацию о проекте, его целях и текущем статусе.

Прецедент «Просмотр открытой финансовой статистики» предназначен для обеспечения прозрачности деятельности фонда. Пользователь может ознакомиться с агрегированной финансовой информацией о поступлениях и расходовании средств.

Прецедент «Просмотр частичной информации о финансовых отчётах» позволяет пользователю получить ограниченную информацию о финансовых отчётах фонда без необходимости регистрации в системе.

Прецедент «Регистрация» предназначен для создания нового пользовательского аккаунта в системе. В рамках данного прецедента пользователь вводит необходимые регистрационные данные и создаёт личный профиль.

Прецедент «Авторизация» предназначен для входа пользователя в систему с использованием ранее созданной учетной записи.

Далее приведено описание прецедентов для роли «Донор»

Прецедент «Создание и редактирование пользовательского профиля» позволяет зарегистрированному пользователю управлять личной информацией, изменять данные профиля и настраивать параметры учетной записи.

Прецедент «Просмотр личной истории пожертвований» предназначен для просмотра пользователем информации о ранее совершённых пожертвованиях, включая дату, сумму и назначение платежа.

Прецедент «Настройка регулярных пожертвований» позволяет пользователю создавать автоматические пожертвования, которые будут осуществляться через заданные промежутки времени.

Прецедент «Разовые пожертвования» предназначен для осуществления однократного пожертвования на выбранный проект или в общий фонд.

Прецедент «Регулярные пожертвования (подписка)» позволяет пользователю оформить подписку на автоматическое перечисление средств фонду с заданной периодичностью.

Прецедент «Целевые пожертвования на конкретный проект» предназначен для поддержки конкретного благотворительного проекта, размещенного на платформе.

Прецедент «Свободные пожертвования в общий фонд» позволяет пользователю перевести средства в общий фонд благотворительной организации без привязки к конкретному проекту.

Прецедент «Просмотр отчётов о расходовании пожертвованных средств» позволяет донору ознакомиться с полной финансовой информацией о том, как были использованы его пожертвования.

Данный прецедент является расширением прецедента «Просмотр частичной информации о финансовых отчётах», поскольку зарегистрированные пользователи имеют доступ к более полной финансовой информации.

Далее приведено описание прецедентов для роли «Администратор»

Прецедент «Управление пользователями системы» предназначен для администрирования пользовательских учетных записей. Администратор может просматривать список пользователей, изменять их данные или ограничивать доступ к системе.

Прецедент «Управление благотворительными проектами» предназначен для создания, редактирования и удаления проектов, размещённых на платформе.

Прецедент «Управление информацией о проектах» позволяет администратору изменять описание проектов, обновлять информацию о ходе реализации и публиковать дополнительные материалы.

Прецедент «Публикация финансовых отчётов» предназначен для размещения финансовых отчетов фонда, содержащих информацию о поступлениях и расходовании средств.

2.3 Информационная модель программного комплекса

Одной из ключевых задач любого современного программного комплекса является эффективное хранение и обработка данных. В большинстве информационных систем для этих целей используются базы данных, позволяющие структурировать информацию, обеспечивать её целостность, а также быстро осуществлять поиск и обработку необходимых данных. Использование баз данных является одним из наиболее надёжных и эффективных способов хранения информации в программных системах.

Разрабатываемое *Web*-приложение для учёта пожертвований в детский фонд предполагает хранение значительного объёма информации, включая данные пользователей, сведения о благотворительных проектах, информацию о пожертвованиях, транзакциях блокчейна, расходах фонда и отчётности. Для хранения и управления данными в рамках программного комплекса используется реляционная база данных.

На основе анализа предметной области и требований к программному комплексу были выделены основные сущности системы, каждая из которых отвечает за хранение определённого типа информации.

К основным сущностям информационной модели относятся:

– пользователи (*Users*);
– кошельки пользователей (*Wallets*);
– благотворительные проекты (*Projects*);
– пожертвования (*Donations*);
– регулярные пожертвования (*Subscriptions*);
– транзакции блокчейна (*BlockchainTransactions*);
– расходы фонда (*Expenses*);

– отчёты по проектам (*Reports*);

– документы и подтверждающие материалы (*Documents*).

Сущность «Пользователи» (*Users*) предназначена для хранения информации о зарегистрированных пользователях системы. Данная сущность содержит основные данные учётной записи пользователя, включая имя, адрес электронной почты, зашифрованный пароль и роль пользователя в системе. Роли позволяют разграничивать права доступа к функционалу системы, например, различать обычных пользователей (доноров) и администраторов фонда.

Сущность «Кошельки пользователей» (*Wallets*) используется для хранения адресов криптовалютных кошельков, подключённых пользователями к системе. Наличие данной сущности позволяет связывать транзакции, выполненные в блокчейн-сети, с конкретными пользователями программного комплекса.

Сущность «Благотворительные проекты» (*Projects*) содержит информацию о проектах фонда, для реализации которых осуществляется сбор пожертвований. В рамках данной сущности хранится информация о названии проекта, его описании, целевой сумме сбора средств, текущем объёме собранных средств и статусе проекта.

Сущность «Пожертвования» (*Donations*) является одной из центральных сущностей информационной модели и предназначена для хранения информации обо всех пожертвованиях, совершённых пользователями системы. Для каждого пожертвования фиксируются данные о пользователе, проекте, сумме пожертвования, способе оплаты, а также статусе транзакции.

Сущность «Регулярные пожертвования» (*Subscriptions*) предназначена для реализации механизма подписки на автоматические пожертвования. Пользователь может настроить периодическое перечисление средств на определённый проект, указав сумму пожертвования и периодичность списания средств.

Сущность «Транзакции блокчейна» (*BlockchainTransactions*) используется для хранения информации о транзакциях, зарегистрированных в блокчейн-сети. В рамках данной сущности фиксируется хэш транзакции, номер блока, используемая сеть блокчейна и статус подтверждения транзакции. Наличие данной сущности позволяет обеспечить прозрачность и проверяемость пожертвований.

Сущность «Расходы фонда» (*Expenses*) предназначена для хранения информации о расходовании средств, собранных в рамках благотворительных проектов. В системе фиксируются данные о сумме расходов, их назначении и связанных подтверждающих документах. Это позволяет обеспечить прозрачность использования средств фонда.

Сущность «Отчёты» (*Reports*) используется для публикации отчётной информации по благотворительным проектам. Отчёты могут содержать текстовое описание результатов реализации проекта, информацию о выполненных работах и итогах использования собранных средств.

Сущность «Документы» (*Documents*) предназначена для хранения ссылок на файлы и подтверждающие материалы, связанные с благотворительными проектами. В качестве таких материалов могут выступать фотографии, отчётные документы, чеки и другие подтверждения использования средств.

На основе выделенных сущностей была разработана структура базы данных программного комплекса. Схема базы данных отражает взаимосвязи между основными сущностями системы и позволяет наглядно представить структуру хранения данных.

В разработанной модели базы данных используются реляционные связи между таблицами, которые обеспечивают целостность данных и позволяют эффективно организовать хранение информации. Основные сущности системы включают пользователей, благотворительные проекты, пожертвования, транзакции блокчейна, расходы фонда и отчётную документацию.

Каждая таблица содержит уникальный идентификатор записи, который используется в качестве первичного ключа и обеспечивает однозначную идентификацию объектов системы. Связи между таблицами реализуются с помощью внешних ключей, позволяющих устанавливать зависимости между данными различных сущностей.

На рисунке 2.3 представлена *ER*-диаграмма базы данных программного комплекса.

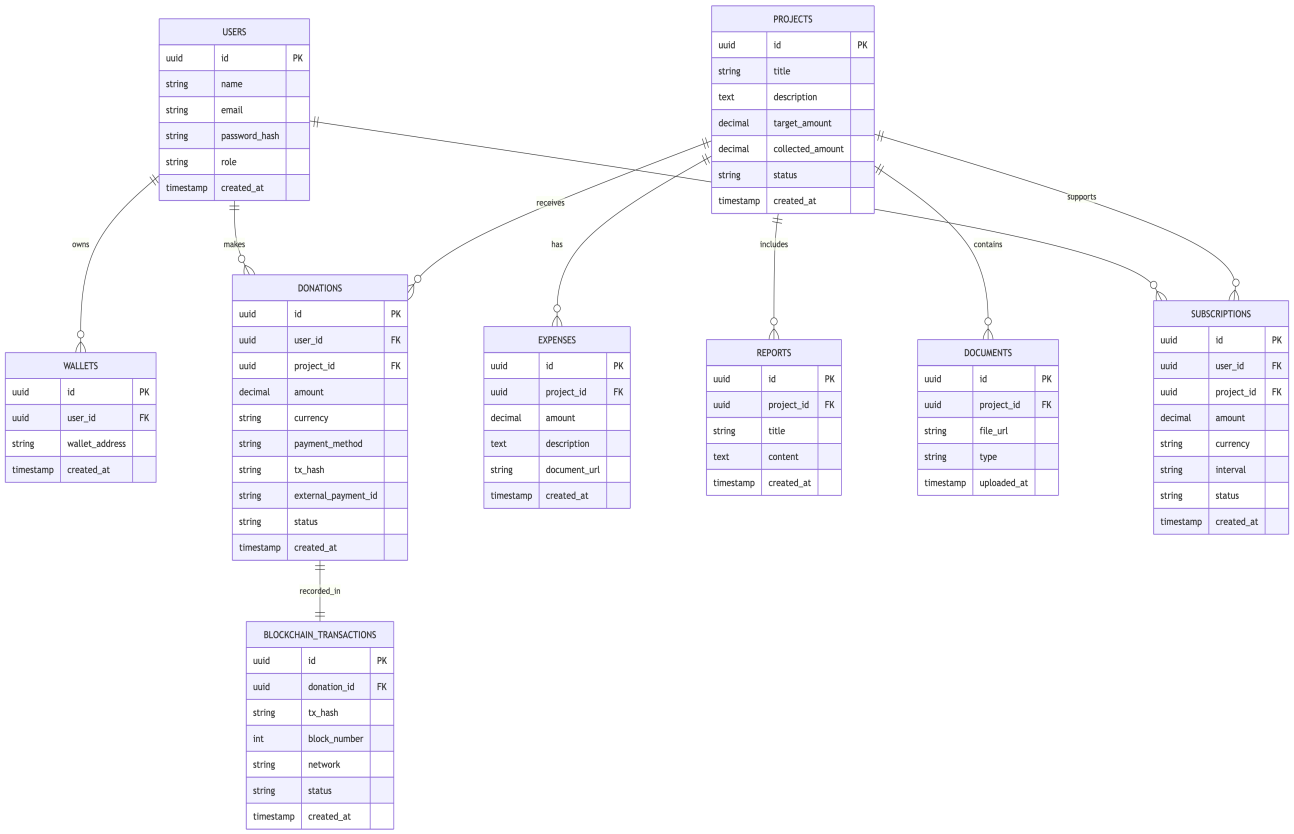

Рисунок 2.3 — *ER*-диаграмма базы данных системы учёта пожертвований

Диаграмма отражает основные сущности системы и связи между ними. Центральной сущностью модели является таблица пожертвований, которая связана с пользователями, благотворительными проектами и транзакциями блокчейна. Такая структура позволяет эффективно отслеживать поступление средств, обеспечивать прозрачность финансовых операций и формировать отчётность по использованию средств фонда.

Таблица 2.1 — *Users*

Название	Тип	Обязательное	Уникальное
id	*UUID*	Да	Да
name	*string*	Да	Нет
email	*string*	Да	Да
password_hash	*string*	Да	Нет
role	*string*	Да	Нет
created_at	*timestamp*	Да	Нет

В таблице *Users* хранится информация о пользователях системы. Каждая запись содержит уникальный идентификатор пользователя, имя, адрес электронной почты, зашифрованный пароль и роль пользователя. Поле role используется для разграничения прав доступа в системе.

Таблица 2.2 — *Wallets*

Название	Тип	Обязательное	Уникальное
id	*UUID*	Да	Да
user_id	*UUID*	Да	Нет
wallet_address	*string*	Да	Да
created_at	*timestamp*	Да	Нет

В таблице *Wallets* хранится информация о криптовалютных кошельках, подключённых пользователями к системе. Каждый кошелёк связан с конкретным пользователем через поле *user_id*. Это позволяет сопоставлять транзакции блокчейна с пользователями системы.

Таблица 2.3 — *Projects*

Название	Тип	Обязательное	Уникальное
id	*UUID*	Да	Да
title	*string*	Да	Нет
description	*text*	Да	Нет
target_amount	*decimal*	Да	Нет
collected_amount	*decimal*	Да	Нет
status	*string*	Да	Нет
created_at	*timestamp*	Да	Нет

В таблице *Projects* хранится информация о благотворительных проектах, для реализации которых осуществляется сбор средств. Для каждого проекта указывается название, описание, целевая сумма сбора, текущая сумма собранных средств и статус проекта.

Таблица 2.4 — *Donations*

Название	Тип	Обязательное	Уникальное
id	*UUID*	Да	Да
user_id	*UUID*	Нет	Нет
project_id	*UUID*	Да	Нет
amount	*decimal*	Да	Нет
currency	*string*	Да	Нет
payment_method	*string*	Да	Нет
tx_hash	*string*	Нет	Да
external_payment_id	*string*	Нет	Нет
status	*string*	Да	Нет
created_at	*timestamp*	Да	Нет

Таблица *Donations* является центральной таблицей системы и содержит информацию обо всех пожертвованиях, совершённых пользователями. В таблице фиксируются данные о пользователе, проекте, сумме пожертвования, способе оплаты и статусе транзакции.

Таблица 2.5 — *BlockchainTransactions*

Название	Тип	Обязательное	Уникальное
id	*UUID*	Да	Да
donation_id	*UUID*	Да	Нет
tx_hash	*string*	Да	Да
block_number	*int*	Нет	Нет
network	*string*	Да	Нет
status	*string*	Да	Нет
created_at	*timestamp*	Да	Нет

В таблице *BlockchainTransactions* хранится информация о транзакциях, зарегистрированных в блокчейн-сети. Для каждой транзакции сохраняется хэш

транзакции, номер блока, используемая сеть блокчейна и статус подтверждения транзакции.

Таблица 2.6 — *Expenses*

Название	Тип	Обязательное	Уникальное
id	*UUID*	Да	Да
project_id	*UUID*	Да	Нет
amount	*decimal*	Да	Нет
description	*text*	Да	Нет
document_url	*string*	Нет	Нет
created_at	*timestamp*	Да	Нет

В таблице *Expenses* хранится информация о расходовании средств, собранных в рамках благотворительных проектов. Данные записи позволяют отслеживать использование средств и обеспечивать прозрачность финансовых операций фонда.

Таблица 2.7 — *Subscriptions*

Название	Тип	Обязательное	Уникальное
id	*UUID*	Да	Да
user_id	*UUID*	Да	Нет
project_id	*UUID*	Да	Нет
amount	*decimal*	Да	Нет
currency	*string*	Да	Нет
interval	*string*	Да	Нет
status	*string*	Да	Нет
created_at	*timestamp*	Да	Нет

В таблице *Subscriptions* хранится информация о подписках пользователей на регулярные пожертвования. Пользователь может настроить автоматическое перечисление средств на определённый проект с заданной периодичностью. Поле *interval* определяет интервал списания средств (например, ежемесячно или ежегодно), а поле *status* определяет текущее состояние подписки.

Таблица 2.8 — *Reports*

Название	Тип	Обязательное	Уникальное
1	2	3	4

1	2	3	4
id	*UUID*	Да	Да
project_id	*UUID*	Да	Нет
title	*string*	Да	Нет
content	*text*	Да	Нет
created_at	*timestamp*	Да	Нет

В таблице *Reports* хранится информация об отчётных материалах, связанных с реализацией благотворительных проектов. Отчёты могут содержать текстовое описание результатов реализации проекта, информацию о проведённых работах и итогах использования собранных средств.

Таблица 2.9 — *Documents*

Название	Тип	Обязательное	Уникальное
id	*UUID*	Да	Да
project_id	*UUID*	Да	Нет
file_url	*string*	Да	Нет
type	*string*	Да	Нет
uploaded_at	*timestamp*	Да	Нет

В таблице *Documents* хранится информация о документах и медиафайлах, связанных с благотворительными проектами. К таким материалам могут относиться фотографии, финансовые документы, чеки, а также иные подтверждения использования средств фонда.